

MediScan AI

AI-Driven Medical Imaging Diagnostic System

Product Requirements Document (PRD)

Version	1.0.0
Date	May 2025
Stack	MERN + Python AI Microservice
Status	DRAFT — Ready for Build

1. Executive Summary

MediScan AI is a full-stack, AI-powered diagnostic assistance platform that enables radiologists and clinicians to upload X-ray, CT scan, and MRI images and receive near-instant, explainable predictions about abnormalities such as tumors, fractures, pneumonia, infections, and other pathologies.

The system combines a ResNet-50-based deep learning backbone with SHAP (SHapley Additive exPlanations) for clinical-grade explainability, and is delivered through a MERN (MongoDB, Express, React, Node.js) web application. A dedicated Python/FastAPI microservice handles all inference workloads, keeping the Node.js backend lightweight and scalable.

The primary goal is to reduce diagnostic error rates and turnaround times in radiology workflows, providing radiologists with a second-opinion tool rather than a replacement.

2. Problem Statement

Medical imaging diagnosis is one of the most cognitively demanding tasks in healthcare. Current challenges include:

- Radiologist fatigue and high image volumes contribute to missed findings, with error rates estimated at 3–5% in routine reads.

- Early-stage abnormalities (small tumors, hairline fractures) are frequently overlooked due to low contrast and image noise.
- Specialist shortages in radiology create bottlenecks, especially in rural and resource-constrained healthcare settings.
- Existing AI tools often operate as black boxes, limiting clinician trust and adoption due to lack of transparency.
- Manual image annotation and reporting workflows are time-intensive, delaying patient care.

MediScan AI directly addresses these gaps by providing an AI-assisted, explainable, and workflow-integrated diagnostic tool.

3. Goals & Success Metrics

3.1 Product Goals

- Achieve a model sensitivity (recall) of $\geq 90\%$ and specificity of $\geq 88\%$ across supported imaging modalities.
- Deliver SHAP-based visual explanations for every prediction, overlaid on the original image.
- Support X-ray, CT scan, and MRI modalities with modality-specific model heads.
- Process a single image and return an annotated result in under 5 seconds under normal load.
- Provide a radiologist-friendly dashboard for case management and report generation.
- Ensure HIPAA-compliant data handling, audit logs, and encrypted storage.

3.2 Success Metrics (KPIs)

Metric	Target	Measurement Method
Model Sensitivity (Recall)	$\geq 90\%$	Test dataset evaluation
Model Specificity	$\geq 88\%$	Test dataset evaluation
AUC-ROC Score	≥ 0.92	ROC curve on held-out set
Inference Latency (P95)	<5 seconds	Application performance monitoring
SHAP Heatmap Generation	100% of predictions	System logging
Radiologist Trust Score	$\geq 4.0 / 5.0$	Post-pilot survey
System Uptime	99.5%	Uptime monitoring (e.g. UptimeRobot)

4. Technology Stack

4.1 Frontend (React.js)

- React.js 18 — Component-based SPA for the radiologist dashboard
- React Router v6 — Client-side routing (Dashboard, Upload, Case History, Reports, Admin)
- Redux Toolkit — Global state management for sessions, cases, and notifications
- Axios — HTTP client for REST API calls
- Tailwind CSS — Utility-first styling for rapid, responsive UI development
- React Dropzone — Drag-and-drop medical image uploader with DICOM/PNG/JPEG support
- Recharts — Confidence score charts and performance analytics dashboards
- React-PDF — Inline viewing and export of diagnostic reports
- React Hot Toast — Non-intrusive feedback notifications

4.2 Backend (Node.js / Express.js)

- Node.js 20 LTS + Express.js 4 — REST API server, middleware orchestration, routing
- JWT (jsonwebtoken) — Stateless authentication tokens with refresh token rotation
- bcrypt — Password hashing with salt rounds
- Multer + AWS S3 (or MinIO self-hosted) — Secure multipart file upload and medical image storage
- Mongoose ODM — Schema-based modeling and query abstraction over MongoDB
- Socket.IO — Real-time progress updates during AI inference pipeline
- Winston + Morgan — Structured request logging and error tracking
- express-validator — Input sanitization and request validation
- Helmet.js + CORS — Security headers and cross-origin policy enforcement
- Bull + Redis — Background job queue for asynchronous inference requests

4.3 Database (MongoDB Atlas)

- MongoDB Atlas (Cloud) or self-hosted MongoDB 7.x
- Collections: users, cases, studies, predictions, reports, audit_logs
- GridFS — Optional large DICOM file storage within MongoDB
- Indexes on patientId, studyDate, modalityType for fast case retrieval

4.4 AI / ML Microservice (Python / FastAPI)

- FastAPI — High-performance async REST framework for inference endpoints
- Python 3.11 — Runtime for all ML workloads
- PyTorch 2.x — Deep learning framework; GPU acceleration via CUDA 12
- ResNet-50 (Pretrained on ImageNet, fine-tuned) — Primary image feature extractor and classification backbone
- torchvision.transforms — Image preprocessing pipeline (resize, normalize, augment)
- SHAP (shap library) — SHapley Additive exPlanations for model interpretability
- GradCAM (pytorch-grad-cam) — Gradient-weighted Class Activation Maps as a secondary explainability method
- scikit-learn — Supplementary evaluation metrics, confusion matrix, ROC/AUC computation
- Pillow (PIL) — Image I/O, preprocessing, and heatmap overlay rendering
- pydicom — DICOM file reading and metadata extraction
- NumPy + Pandas — Array operations and dataset management
- OpenCV (cv2) — Image resizing, normalization, and augmentation pipeline
- Uvicorn — ASGI server for FastAPI production deployment

4.5 Explainability (XAI) Layer

- SHAP DeepExplainer — Computes Shapley values on deep neural network layers to quantify each pixel group's contribution to the prediction
- SHAP GradientExplainer — Fast gradient-based Shapley approximation for high-resolution images
- GradCAM / GradCAM++ — Produces class-discriminative localization heatmaps by flowing gradients back to the final convolutional layer of ResNet-50
- Overlay Rendering — Python + Pillow combines SHAP/GradCAM heatmaps with original images, outputting annotated PNG for the frontend
- Confidence Score Decomposition — Per-class probability scores (Normal / Abnormal / Sub-type) surfaced to the UI

4.6 DevOps & Infrastructure

- Docker + Docker Compose — Containerization of all services (React, Node, Python, MongoDB, Redis)
- NGINX — Reverse proxy, SSL termination, and static file serving
- GitHub Actions — CI/CD pipeline: lint, test, build, push Docker image, deploy
- AWS EC2 / DigitalOcean Droplet — Cloud hosting for production workloads
- AWS S3 or MinIO — Medical image object storage with lifecycle policies
- Let's Encrypt (Certbot) — Free TLS/SSL certificates
- Prometheus + Grafana — Infrastructure and API performance monitoring

5. System Architecture

5.1 High-Level Architecture

MediScan AI follows a microservices-inspired layered architecture with four primary tiers:

1. Presentation Layer — React SPA served via NGINX. All API calls go through /api proxy.
2. Application Layer — Node.js/Express REST API. Handles auth, business logic, file orchestration, and database operations. Dispatches inference jobs to the AI microservice via internal HTTP or job queue.
3. AI Inference Layer — Python/FastAPI microservice (isolated container). Receives image path or base64 payload, runs ResNet-50 inference, computes SHAP values, generates annotated heatmaps, returns structured JSON.
4. Data Layer — MongoDB for relational case/user data; Redis for session cache and job queues; S3/MinIO for binary image storage.

5.2 Data Flow

5. Radiologist logs in → JWT issued by Node.js auth service.
6. User uploads image (DICOM/PNG/JPEG) via React Dropzone → Multer stores to S3, inserts study record in MongoDB.
7. Node.js publishes an inference job to Bull/Redis queue with study_id and image_url.
8. Python FastAPI worker picks up job → downloads image from S3 → preprocesses (resize 224x224, normalize) → passes through ResNet-50.
9. SHAP DeepExplainer / GradCAM runs on the prediction → heatmap rendered and uploaded to S3 as annotated_<study_id>.png.
10. Inference results (class label, confidence scores, heatmap_url, shap_values summary) saved to predictions collection in MongoDB.
11. Socket.IO pushes real-time completion event to the radiologist's browser.
12. React dashboard fetches results, renders annotated image + confidence breakdown.
13. Radiologist reviews, adds notes, and exports PDF report.

6. AI Model Details

6.1 Base Model: ResNet-50

- Architecture: 50-layer Residual Network with skip connections, pre-trained on ImageNet-1K (1.2M images, 1000 classes).
- Transfer Learning Strategy: Freeze all layers except the last two residual blocks and the fully connected head during initial training. Unfreeze all layers for fine-tuning at a low learning rate (1e-5).

- Custom Classification Head: Replace default FC(1000) with FC(512) → ReLU → Dropout(0.4) → FC(num_classes). num_classes varies by modality.
- Input Shape: $3 \times 224 \times 224$ (RGB; DICOM grayscale frames are channel-replicated).

6.2 Modality-Specific Task Heads

Modality	Target Conditions	Output Classes
X-Ray (CXR)	Pneumonia, Cardiomegaly, Pleural Effusion, COVID-19, Normal	5-class softmax
CT Scan	Pulmonary nodules, Hemorrhage, Liver lesions, PE, Normal	6-class softmax
MRI	Brain tumor (Glioma, Meningioma, Pituitary), Normal	4-class softmax

6.3 Training Details

- Dataset: NIH ChestX-ray14 (X-ray), RSNA Brain MRI dataset, LUNA16 / COVID-CT (CT). Augmented with TensorFlow/Albumentations: random horizontal flip, rotation ($\pm 15^\circ$), brightness jitter, Gaussian noise.
- Loss Function: Cross-Entropy with class-weighted sampling to handle class imbalance.
- Optimizer: AdamW with weight decay $1e-4$; cosine annealing LR scheduler.
- Batch Size: 32; Epochs: 50 with early stopping (patience=7) on validation loss.
- Evaluation: Accuracy, Sensitivity, Specificity, F1 Score, AUC-ROC (per class and macro-averaged).

6.4 XAI: SHAP Implementation

- SHAP DeepExplainer: Uses a background dataset of 100 randomly sampled training images as baseline. Computes Shapley values for all spatial locations.
- SHAP GradientExplainer: Alternative fast path using integrated gradients, used when DeepExplainer is too slow (CT/MRI volumes).
- Output: Per-pixel SHAP contribution matrix → normalized to $[0,1]$ → matplotlib colormap (red = high contribution, blue = low) → alpha-blended with original grayscale image.
- GradCAM Fallback: When SHAP computation exceeds 3s threshold, GradCAM++ is used as real-time fallback.

7. Functional Requirements

FR-01: User Authentication & Roles

- Support three roles: Admin, Radiologist, Referring Physician.
- Login via email + password; JWT access token (15 min) + refresh token (7 days) rotation.
- Admin can create/suspend users, manage hospital settings, and view audit logs.

FR-02: Medical Image Upload

- Accept DICOM (.dcm), PNG, JPEG, and TIFF formats.
- Support batch upload of up to 50 images per study.
- Auto-detect modality from DICOM metadata tags (0008,0060). Manual override available.
- Max file size: 100 MB per image; 500 MB per batch.

FR-03: AI Inference Engine

- Route uploaded images to correct modality-specific ResNet-50 model head.
- Return: predicted class label, confidence score per class, annotated heatmap image URL.
- Asynchronous processing with real-time Socket.IO progress events: Queued → Processing → Complete / Failed.

FR-04: Explainability (XAI) Overlay

- Render SHAP heatmap overlaid on original image with adjustable opacity slider.
- Display top-3 contributing regions with textual clinical labels (e.g., “Right lower lobe opacity”).
- Toggle between SHAP and GradCAM views.

FR-05: Case Management Dashboard

- Radiologist home screen lists all pending, in-review, and completed studies.
- Filter by modality, date range, urgency flag, and assigned physician.
- Case detail view: original image, AI overlay, confidence charts, past studies for same patient.

FR-06: Report Generation

- Auto-generate structured PDF diagnostic report: patient demographics, study metadata, AI findings, heatmap image, radiologist notes.

- Radiologist can edit AI-generated text before finalizing report.
- One-click download as PDF; optional email delivery to referring physician.

FR-07: Audit & Compliance

- Every prediction, login, file access, and report export is logged to audit_logs collection with timestamp, user_id, action, and IP address.
- Logs are immutable (append-only). Exportable as CSV for compliance review.

8. Non-Functional Requirements

Category	Requirement	Detail
Performance	Inference <5s (P95)	Optimize with ONNX Runtime or TorchScript export; GPU inference preferred
Scalability	Horizontal scaling	AI microservice stateless; scale via Docker Swarm / Kubernetes HPA
Security	HIPAA-aligned	AES-256 at rest, TLS 1.3 in transit, field-level encryption for PHI
Availability	99.5% uptime	Load balancer health checks; auto-restart on container failure
Usability	Mobile-responsive	Tailwind breakpoints; minimum 1024px optimized for radiology workstations
Accessibility	WCAG 2.1 AA	Semantic HTML, ARIA labels, sufficient color contrast on heatmaps
Auditability	Full audit trail	Every DB write tagged with actor, timestamp, IP; stored in MongoDB
Data Retention	7-year image archive	S3 lifecycle policies; MongoDB TTL index for session data only

9. API Contract (Key Endpoints)

9.1 Node.js REST API (Base: /api/v1)

Method	Endpoint	Description
POST	/auth/login	Authenticate user, return JWT access + refresh tokens
POST	/auth/refresh	Exchange refresh token for new access token

Method	Endpoint	Description
POST	/studies/upload	Upload image(s), create study record, enqueue inference job
GET	/studies/:id	Fetch study metadata + latest prediction results
GET	/studies	List all studies (paginated, filterable)
GET	/predictions/:studyId	Fetch AI prediction, confidence scores, heatmap URL
POST	/reports/:studyId	Finalize and generate PDF report for a study
GET	/reports/:studyId/download	Download finalized PDF report
GET	/admin/audit-logs	Admin only: retrieve paginated audit log entries
GET	/admin/users	Admin: list all users with role and status

9.2 Python FastAPI AI Microservice (Base: /inference)

Method	Endpoint	Description
POST	/predict	Accept {image_url, modality} → return {label, confidence, shap_url}
POST	/predict/batch	Batch inference for multiple images in one study
GET	/health	Service health check (used by Node.js before dispatching)
GET	/models	List loaded models and their version, accuracy metadata

10. Database Schema (MongoDB)

Key Collections

- users: { _id, name, email, passwordHash, role, hospitalId, createdAt, isActive }
- studies: { _id, patientId, patientName, dob, modalityType, uploadedBy, imageUrls[], studyDate, status, createdAt }
- predictions: { _id, studyId, modelVersion, label, confidence{}, heatmapUrl, shapValues{}, gradcamUrl, processingTimeMs, createdAt }
- reports: { _id, studyId, radiologistId, aiSummary, radiologistNotes, pdfUrl, finalizedAt, createdAt }
- audit_logs: { _id, actorId, action, targetId, targetType, ipAddress, userAgent, timestamp }

11. Project Directory Structure

Path	Purpose
mediscan-ai/	Monorepo root
client/	React.js frontend (CRA or Vite)
server/	Node.js / Express backend
server/routes/	Express route files (auth, studies, reports, admin)
server/models/	Mongoose schemas
server/middleware/	Auth guard, error handler, file upload
server/queues/	Bull queue producer / consumer
ai-service/	Python FastAPI microservice
ai-service/models/	Saved .pt model weights per modality
ai-service/inference/	ResNet-50 forward pass, SHAP, GradCAM logic
ai-service/training/	Training scripts, dataset loaders, eval notebooks
docker-compose.yml	Orchestrates all services
nginx/	NGINX reverse proxy config
.github/workflows/	CI/CD pipeline YAML files

12. UI/UX Screen Inventory

Screen	Key Components
Login / Register	Email + password form, role selection, forgot password flow
Dashboard Home	Study statistics cards, recent cases table, pending queue count
Upload Study	Drag-and-drop zone, modality selector, patient ID form, bulk upload progress
Study Detail / Analysis	Original image viewer, AI heatmap overlay, opacity slider, confidence bar chart, SHAP/GradCAM toggle, notes textarea
Case History	Timeline of all studies for a patient, comparison viewer for two time-points
Report Preview & Export	Structured PDF preview, editable text fields, download + email buttons
Admin Panel	User management table, system stats, audit log viewer, model version info
Settings / Profile	User profile, notification preferences, hospital details

13. Development Phases & Milestones

Phase	Name	Deliverables	Duration
1	Foundation	Repo setup, Docker Compose, MongoDB + Redis, NGINX, CI/CD skeleton, Auth module (register/login/JWT)	2 weeks
2	AI Microservice	FastAPI scaffold, ResNet-50 model loading, single-image /predict endpoint, SHAP integration, GradCAM fallback, health check	3 weeks
3	Core Backend	Study upload to S3, Bull job queue, prediction result storage, Socket.IO events, report PDF generation (Puppeteer or pdfkit)	2 weeks
4	Frontend MVP	Login, Dashboard, Upload Study, Study Detail with heatmap viewer, Confidence chart	3 weeks
5	Advanced Features	Case History, Report Export, Batch upload, Comparison viewer, Admin Panel, Audit Logs	2 weeks
6	Testing & QA	Unit tests (Jest, pytest), Integration tests, Performance benchmarks, Security audit, UAT with radiologists	2 weeks
7	Deployment	Production Docker images, NGINX SSL, S3 lifecycle, Monitoring dashboards, Documentation	1 week

14. Security & Compliance

- HIPAA Safeguards: All PHI encrypted at rest (AES-256 on MongoDB Atlas + S3 SSE) and in transit (TLS 1.3).
- Authentication: bcrypt password hashing (cost factor 12); JWT with short expiry; refresh token rotation with revocation list in Redis.
- Authorization: Role-based access control (RBAC) middleware on every protected Express route.
- Input Validation: express-validator on all API inputs; Multer file type whitelist (.dcm, .png, .jpg, .tiff only).
- Rate Limiting: express-rate-limit on auth and upload endpoints to prevent brute-force and DoS.
- Audit Logging: Every action logged with actor, IP, timestamp to immutable MongoDB collection.
- Data Anonymization: Option to strip DICOM metadata (patient name, DOB) before inference for de-identified workflows.

- Dependency Scanning: npm audit + pip-audit in CI pipeline; Snyk integration for vulnerability alerts.

15. Testing Strategy

Unit Testing

- Node.js: Jest + Supertest for route handlers, middleware, and service functions.
- Python: pytest + httpx for FastAPI endpoint tests; unittest for model preprocessing functions.

Integration Testing

- Full upload-to-prediction flow tested end-to-end against a test MongoDB instance.
- Socket.IO event delivery verified with socket.io-client in Jest tests.

Model Evaluation

- Held-out test split (20%) evaluated on Accuracy, F1, AUC-ROC per class.
- Confusion matrix generated post-training for each modality head.
- SHAP output visually validated by a radiology consultant for clinical relevance.

Performance Testing

- Apache JMeter load test: 50 concurrent uploads; measure P50/P95/P99 inference latency.
- GPU vs CPU inference benchmarked; ONNX export evaluated for latency reduction.

16. Open Questions & Future Scope

Open Questions

- DICOM viewer: Integrate Cornerstone.js for in-browser DICOM rendering (Phase 5 candidate)?
- Multi-institution: Should the system support multi-tenant hospital isolation in v1, or post-launch?
- Model versioning: Use MLflow or DVC for experiment tracking and model artifact versioning?

Future Scope (Post-v1)

- 3D CT/MRI volume analysis using 3D ResNet or ViT (Vision Transformer) architectures.
- Federated Learning support for training across hospitals without sharing raw patient data.
- Real-time collaboration: multiple radiologists annotating the same study simultaneously.
- LLM-powered narrative report generation (GPT-4o / Claude API) from structured prediction data.
- Mobile app (React Native) for on-call radiologist access.
- HL7 FHIR / PACS integration for direct hospital system connectivity.

17. Glossary

Term	Definition
ResNet-50	50-layer deep residual neural network; achieves high accuracy on image classification tasks via skip connections.
SHAP	SHapley Additive exPlanations: a game-theoretic approach to explain AI model output by attributing each feature's contribution.
GradCAM	Gradient-weighted Class Activation Mapping: highlights regions of an image that most influenced a CNN's prediction.
DICOM	Digital Imaging and Communications in Medicine: standard format for medical imaging data including metadata.
AUC-ROC	Area Under the Receiver Operating Characteristic Curve: summary metric for binary/multi-class classifier performance.
MERN	MongoDB, Express.js, React.js, Node.js: a popular full-stack JavaScript framework combination.
PHI	Protected Health Information: individually identifiable health information regulated under HIPAA.
Bull	A Redis-based Node.js queue library for managing background jobs and async task processing.
FastAPI	High-performance Python web framework for building APIs with automatic OpenAPI documentation.
HIPAA	Health Insurance Portability and Accountability Act: US law governing protection of patient health data.

End of Document

MediScan AI — PRD v1.0.0 — May 2025